EVENTLY BACKEND · SNAPSHOT 9 NOV 2026

How Evently fits together

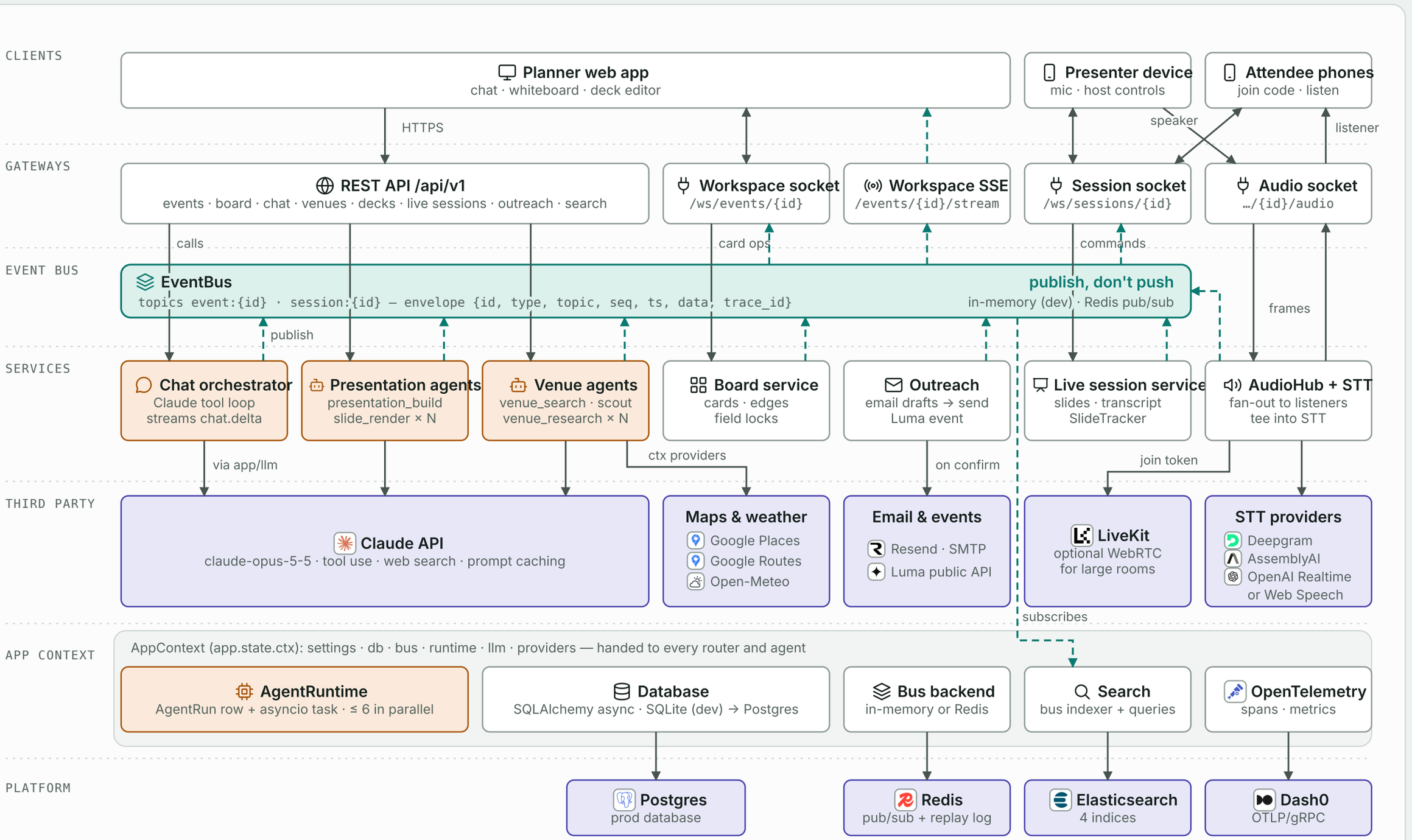
One FastAPI service. A planner chats with Claude, agents research venues and build decks on a live whiteboard, and a presenter's voice moves the slides on every attendee's phone. Four views of the same system.

1 System overview · 2 Agent fan-out · 3 Live session · 4 Platform & integrations

LEGEND → request or call · ⇢ envelope on the EventBus · agent run (AgentRun) · agent bound to its card · outside this codebase · planned, not built

1 · System overview

How to read: top to bottom. Requests travel down from the clients; state changes travel back up as teal envelopes, and the gateways only forward them.

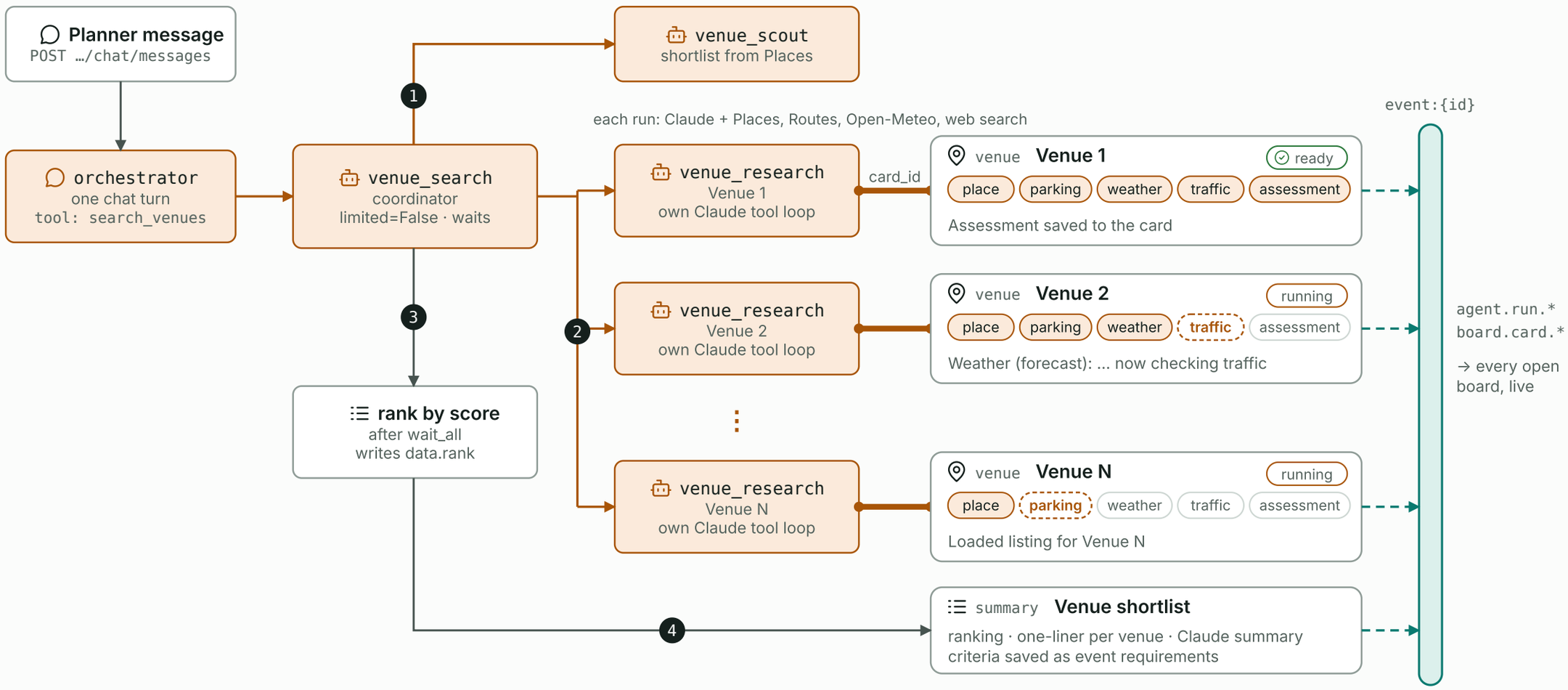


Publish, don't push: services never write to a socket. Every state change becomes an envelope on `event:{id}` or `session:{id}`, and the workspace socket, SSE stream and session socket only forward it (with `?since=` replay). Raw audio is the one stream that skips the bus: the AudioHub hands frames straight to the audio socket.

2 · Agent fan-out

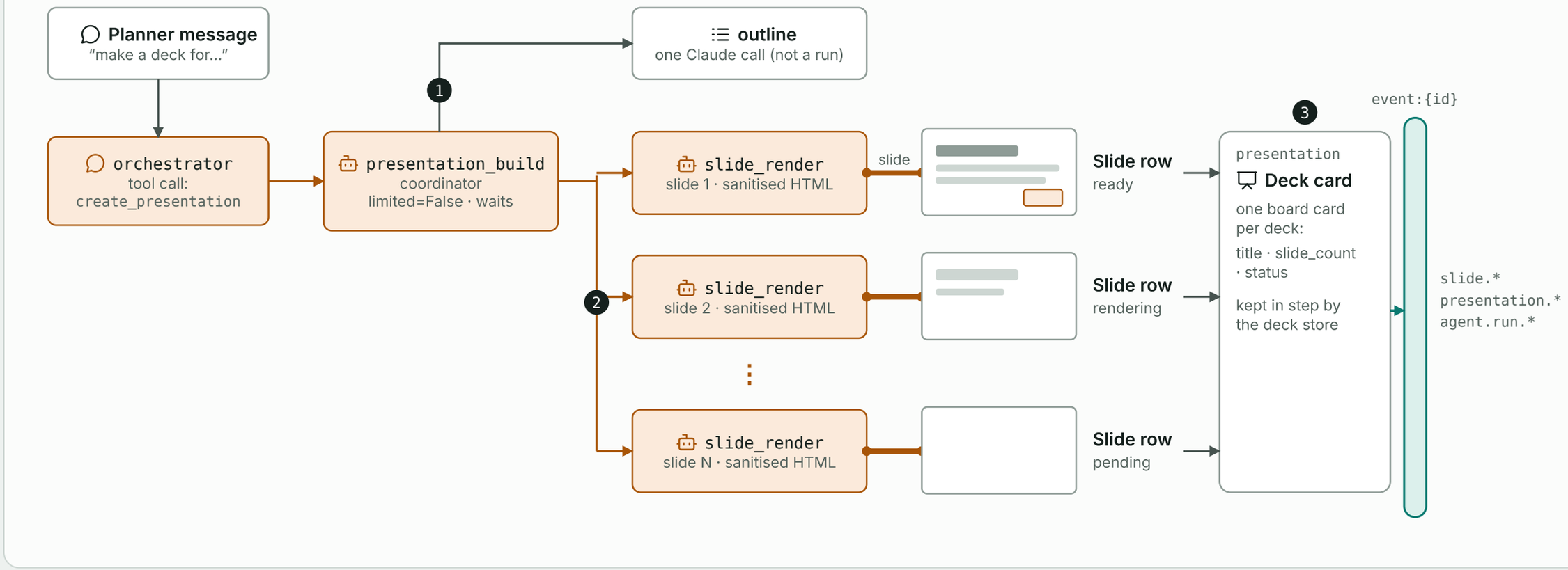
How to read: left to right. Amber boxes are real running agents. The thick amber link ties each agent to the one card or slide it fills, so one card on the whiteboard is one running agent.

Venue search



1. The coordinator runs one scout, which only shortlists real places from the Places provider.
2. For each candidate it creates a venue card and spawns one `venue_research` run bound to it. Runs share a pool of parallel slots (6 by default); each tool result writes its section to the card the moment it returns.
3. After every child finishes, the coordinator ranks the cards by score.
4. It writes the “Venue shortlist” summary card and the orchestrator posts a follow-up chat note.

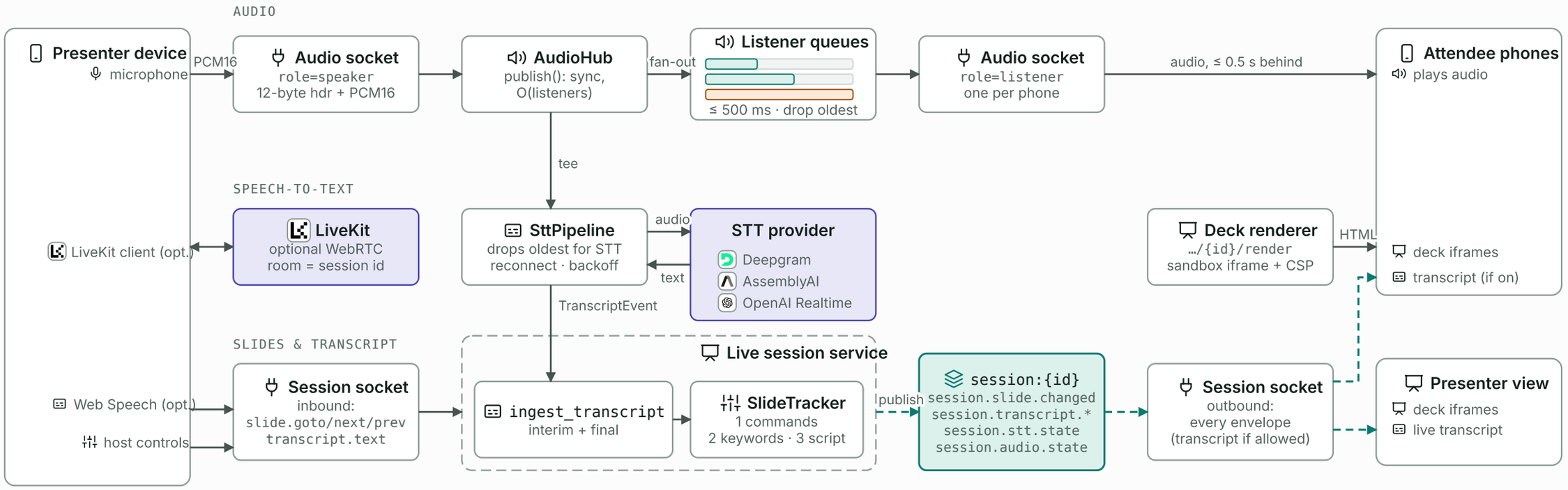
Deck build



Same shape: (1) one outline call, (2) one `slide_render` run per slide, each filling its own Slide row with sanitised HTML, (3) the deck store keeps a single deck card in step and publishes `slide.*` and `presentation.*`.

3 · Live session

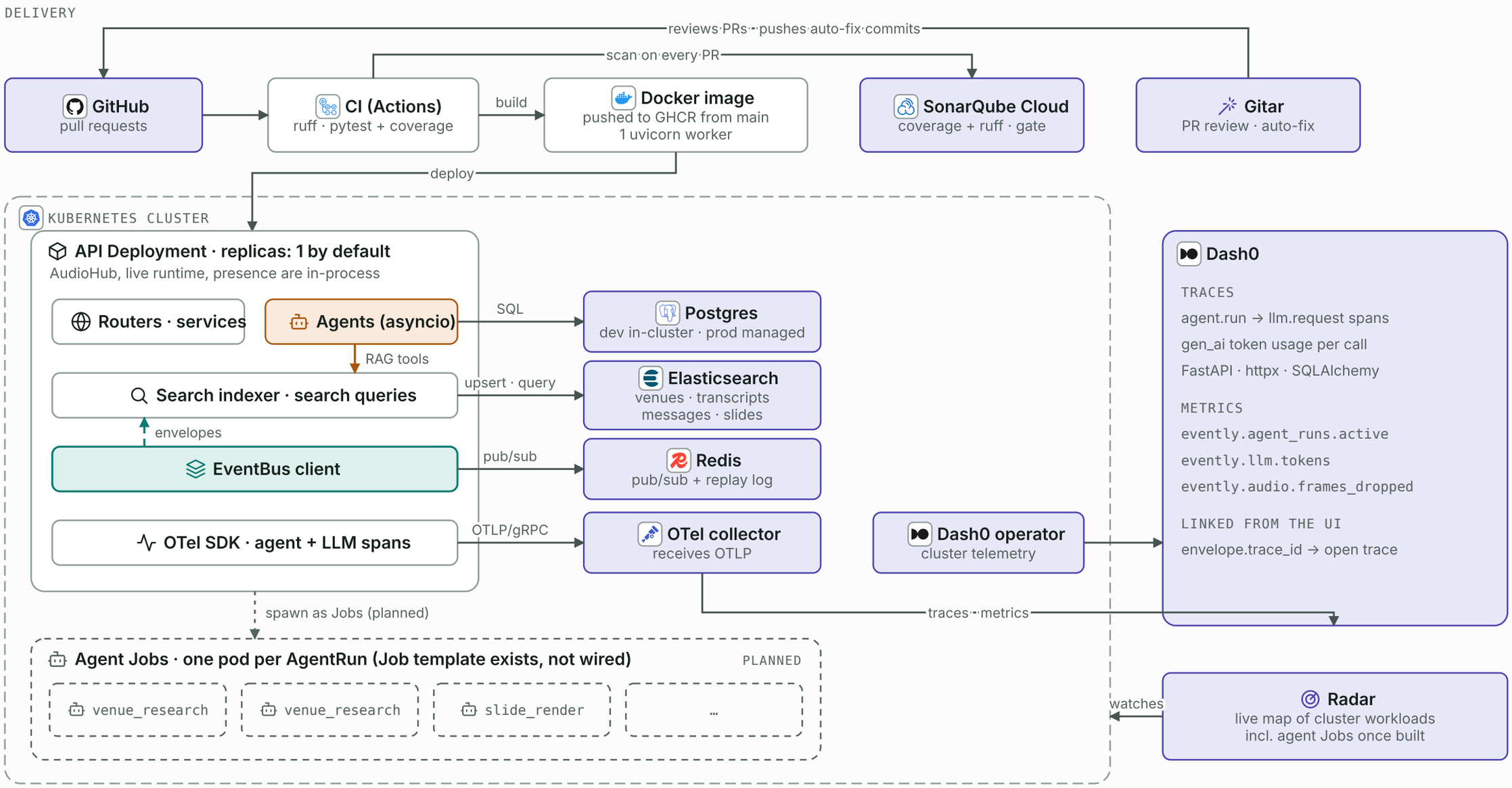
How to read: three lanes. Audio on top, speech-to-text in the middle, slide decisions at the bottom.
The presenter appears on both ends: as the source on the left and as a viewer on the right.



The audio path never waits. Each listener's queue holds at most 500 ms; when a phone falls behind, its oldest frames are dropped and counted. The STT tee also drops old audio rather than block, and if the provider fails for good the broadcast continues without a transcript. The SlideTracker prefers precision: voice commands win, then the next slide's keywords, then script coverage. Every slide renders alone in a sandboxed `srcdoc` iframe under a strict CSP.

4 · Platform & integrations

How to read: delivery across the top, the cluster below, hosted services on the right. Dashed grey is planned. Manifests live in `deploy/k8s`.



The API runs as one replica by default because the AudioHub, live-session runtime and presence live in process memory; an optional scale-out component adds an autoscaler and sticky WebSocket routing, and the Redis bus lets envelopes cross pods. Postgres and Elasticsearch run in the cluster in dev and as managed services in prod. The indexer subscribes to the bus and upserts venue cards, transcript segments, chat messages and slides into four Elasticsearch indices; search routes answer 503 until the cluster is reachable. Telemetry is off unless enabled, and then every envelope carries the `trace_id` of the work that produced it.

PLANNED

Agent Jobs: one Kubernetes pod per agent run instead of an asyncio task, so Radar shows a pod per whiteboard card. A Job template exists in `deploy/k8s/jobs`; the runtime does not launch it yet.

BUILT

RAG tools: the orchestrator calls `search_venue_memory` (past venue research) and `search_event_history` (talks, chat, slides). Without Elasticsearch they answer “not available”.

Drawn from `docs/IMPLEMENTATION_PLAN.md` and the `app/` modules: `main.py`, `core/context.py`, `realtime/events.py`, `agents/runtime.py`, and the agents, live, transcription, integrations, providers, search and observability packages.